

WIRA AI WhatsApp Recruitment Chatbot – Design & Implementation Document

Introduction

White-Force's **WIRA AI** chatbot is an AI-powered recruitment assistant that helps job-seekers discover relevant roles and guides them through the screening process. The company now wants to extend this assistant to **WhatsApp** so that anyone who saves the company's WhatsApp number and sends a message can instantly start a conversation, search for jobs by skill/location and provide screening information. This document describes the design, architecture and implementation plan for building a **WhatsApp recruitment chatbot** integrated with WIRA AI and a recruitment system.

Objectives

- Provide an intuitive WhatsApp chat experience for candidates where they can say “hello” and immediately start interacting with WIRA AI to search jobs and apply.
- Collect candidate details (name, experience, location, skills, salary expectations, notice period, résumé) through conversational screening and store them in a database/ATS.
- Leverage WIRA AI's embedding-based job matching engine to suggest relevant jobs and rank candidates.
- Ensure the solution is production-ready: secure, scalable and multi-tenant so it can be offered as a SaaS service.

WIRA AI overview

WIRA AI is an internal AI assistant used by White-Force. It already lives on the company's website and uses embedding models and a job dataset to answer job-related questions, display available vacancies and perform preliminary screening. Candidates can ask for roles by skill or location, and the AI returns matching openings with links to apply. The system collects information (experience, skills, location, salary expectations, résumé) and stores it in the company's applicant tracking system. Embeddings allow WIRA AI to compute similarity between a candidate's profile and job descriptions, producing relevance scores and personalized recommendations.

WhatsApp Cloud API Integration

To enable a chatbot on WhatsApp you need access to the **WhatsApp Business Platform**. Meta's documentation lists the following steps and prerequisites:

- **Developer & business accounts** – you must register as a developer and create a Meta app with the “Connect on WhatsApp” use case. The Quickstart page guides you through creating the app, connecting it to a WhatsApp Business Account (WABA) and obtaining your phone-number ID. You'll also need a verified Meta Business Manager with a display name and business documentation; this process typically takes **2–7 business days**.

- **Send first message & webhooks** – after connecting your app to a WABA you can generate a temporary access token, pick a “from” phone number and send a test message. Meta’s Quickstart describes how to set up a webhook endpoint to receive incoming messages. The webhook receives message payloads containing the sender’s WhatsApp ID, message type, timestamp and content.
- **Permanent access tokens** – temporary tokens expire quickly; you must create a **system user** and generate a permanent access token for production.
- **Pricing** – Meta charges per conversation: marketing messages cost \$0.025–0.14, utility messages cost \$0.004–0.05 and customer-initiated service messages are free within the 24-hour window. Each business gets 1 000 free service conversations per month.
- **BSP vs direct API** – you can obtain WhatsApp API access either through a Business Solution Provider (Infobip, Twilio, etc.) or directly from Meta. A BSP manages infrastructure, compliance and integrations; direct API access offers full control but requires more engineering work.

Meta setup workflow

1. **Create Meta app & WABA** – register on the Meta for Developers portal, create a business-type app and choose “Connect on WhatsApp” as the use case. Link your business portfolio or create a new one; Meta will provision a WhatsApp Business Account.
2. **Add phone number & send test message** – on the API Setup page, add or select a phone number, generate a temporary access token and send a “hello_world” template message to your test number. Reply back to keep the conversation open.
3. **Configure webhooks** – host a publicly accessible HTTPS endpoint. Configure the callback URL and verify token in the Meta dashboard. When a user sends a message, Meta posts a JSON payload to your webhook.
4. **Generate permanent token** – in Business Settings create a system user, assign “WhatsApp Business Messaging” permissions and generate a long-lived access token. Store tokens securely (environment variables or secrets manager).

Architecture

The chatbot requires a backend server to manage WhatsApp sessions, forward messages to WIRA AI and persist candidate data. The high-level architecture is shown below.

Components

Component	Description
WhatsApp Messenger	Candidates send messages from their phone; no installation beyond WhatsApp required.
Meta Cloud API	Receives candidate messages and forwards them via HTTPS to your webhook; provides REST API for sending messages.
Webhook Server	A backend (e.g., Node.js with NestJS) that verifies signatures, manages sessions and dispatches messages. It sends candidate messages to the conversation engine and responds with replies.

Conversation Engine & WIRA AI	Contains business logic. It determines if the user is new, collects details through screening questions, calls WIRA AI's embedding-based API to search jobs and computes match scores.
Database & ATS	Stores user profiles, conversation history, candidate data, job listings, application status and tenant info. PostgreSQL and Redis are recommended for persistence and caching.
Recruiter Dashboard	A web application (React/Next.js) where recruiters view candidate profiles, download résumés, see conversation transcripts and review match scores.

Backend Implementation

1. **Webhook server:** The server must expose a **POST /webhook/whatsapp** endpoint. It verifies the **X-Hub-Signature** header to ensure the request came from Meta. It then parses the message payload (sender ID, message type and content). For new users it creates a session and sends a welcome message; for existing users it retrieves the session state from Redis.
2. **Conversation logic:** Implement a finite-state machine to handle screening. For example:
 - Ask for **name, years of experience, skills, location, current salary/CTC, expected salary, notice period** and **résumé**. Use reply buttons or list messages to collect structured responses where possible.
 - Validate inputs and ask follow-up questions. Persist answers after each step.
 - When all required fields are collected, call WIRA AI's API to retrieve matching jobs and compute match scores.
3. **WIRA AI integration:** WIRA AI exposes an HTTP API (to be provided by the internal AI team) that accepts a user query and candidate profile and returns relevant jobs ranked by similarity. Use secure API keys; set appropriate timeouts and retry logic. For example:

```
POST /api/wira/search
Content-Type: application/json
Authorization: Bearer <WIRA_API_KEY>
{
  "query": "java developer in Mumbai",
  "candidate": { "skills": ["Java", "Spring"], "experience": 4, "location": "Mumbai" }
}
```

The response contains job IDs, titles, companies and similarity scores which are sent back to the user via WhatsApp.

4. **Data storage:** Use PostgreSQL for structured data and Redis for session caching. Suggested tables:
 - **users** (id, whatsapp_id, name, created_at, tenant_id)
 - **candidates** (user_id, experience, skills, location, current_ctc, expected_ctc, notice_period, resume_url, created_at, tenant_id)
 - **jobs** (id, title, description, location, department, company, created_at, tenant_id)
 - **applications** (candidate_id, job_id, match_score, status, created_at, tenant_id)

- **conversations** (user_id, message, direction, timestamp, tenant_id)
Each table includes a **tenant_id** to support multi-tenant SaaS use.
- 5. **Resume upload & parsing:** WhatsApp allows users to send documents; the webhook receives a media ID. Use the Cloud API to download the file, store it in S3/Azure Blob and run it through a résumé parser (e.g., OpenAI, Gemini or open-source models). Extract skills, education, experience and contact details to enrich the candidate profile.
- 6. **Job matching engine:** Combine WIRA AI's embeddings with structured filtering. For each candidate, compute similarity between their skill vector and job descriptions. Factor in location and experience. Provide top N matches along with match percentages. Store match scores in the **applications** table for recruiter review.

Candidate Experience Flow

1. **Initiation:** Candidate sends “Hi” or any message to the company’s WhatsApp number. The chatbot replies with a welcome message explaining its capabilities and asks for the candidate’s preferred language.
2. **Screening:** The bot asks successive questions to collect name, experience, skills, location and other details. For free-text questions, the conversation engine uses NLP to extract relevant entities. The candidate can upload a résumé as a PDF or DOC, which the bot acknowledges and parses.
3. **Job search & recommendation:** Once sufficient information is collected, the bot queries WIRA AI’s search API. It returns a list of matching jobs with titles, companies and short descriptions. The bot sends interactive messages with buttons for each role so the candidate can see details or apply.
4. **Application:** If the candidate chooses to apply, the bot confirms their interest and stores an application record. Optional: send a WhatsApp template message to confirm application submission.
5. **Follow-up:** The bot can schedule interviews, send reminders or answer FAQs. Candidates can return to the chat to ask additional questions anytime within the 24-hour service window. Beyond 24 hours, a message template is needed to re-engage.

Recruiter Dashboard & ATS Integration

Recruiters need a web interface to view candidate information and track applications. The dashboard should provide:

- **Candidate list** – search and filter by skills, experience, location, status and match score.
- **Profile details** – view conversation history, screening answers and extracted résumé data.
- **Job matches** – see which jobs were suggested by WIRA AI, candidate match scores and application status.
- **Résumé download** – link to the résumé file stored in S3/Blob.
- **Multi-tenant separation** – show only data belonging to the logged-in tenant (company). Implement role-based access control for admins, recruiters and managers.

Production & SaaS-level Considerations

1. **Security** – All traffic between WhatsApp, your server and WIRA AI must be encrypted (HTTPS/TLS). Store API keys and tokens in a secrets manager; never hard-code them.

Verify webhooks using the **X-Hub-Signature**. Implement rate limiting and input sanitization to prevent abuse.

2. **Scalability** – Use horizontal scaling (multiple webhook instances behind a load balancer) and a message queue (BullMQ/RabbitMQ) for CPU-intensive tasks like résumé parsing and AI calls. Use Redis for caching sessions and limiting concurrency. Host on reliable cloud infrastructure (AWS, Azure or GCP). For multi-tenant SaaS, include **tenant_id** in each request and enforce row-level security.
3. **Compliance** – Adhere to data protection regulations. Provide a privacy policy explaining how candidate data will be used and stored. Allow candidates to delete their data on request.
4. **Monitoring & logging** – Instrument the backend to capture metrics (requests per second, response times, error rates). Use tools like Prometheus and Grafana or SaaS platforms such as Datadog. Log all interactions for audit and debugging.

Conclusion & Next Steps

Building a WIRA AI-powered recruitment chatbot on WhatsApp involves more than just sending messages. You must set up a verified Meta Business account, implement a secure webhook, handle conversational state, integrate with WIRA AI for job search and ranking, store candidate information and provide recruiters with a dashboard. Following the guidelines above and leveraging Meta's documentation ensures a robust foundation. Once the core system is in place, the team can iterate on conversation design, add multilingual support, integrate calendar scheduling and use analytics to continuously improve candidate experience.